

Highly Intelligent Volumetric Entities

Eric Cardozo
Rensselaer Polytechnic Institute
cardoe@rpi.edu

Peter Chen
Rensselaer Polytechnic Institute
chenk14@rpi.edu

Matthew McCall
Rensselaer Polytechnic Institute
mccalm@rpi.edu

Abstract—This paper presents highly intelligent volumetric entities and the systems used to design, implement, and evaluate them.

Index Terms—volumetric entities, simulation, robotics, Python, ROS2, Gazebo

I. INTRODUCTION

Aerial swarms have attracted significant research interest because of their potential applications in multi-robot missions [1]. However, many swarm demonstrations remain limited in scale and are often studied in controlled settings rather than fully unstructured real-world environments [1]. In unpredictable situations such as collapsed buildings or wildfires, a centralized system can suffer from many problems, such as single point of failure and high latency, which could take a few drones out, and in the worst case, the whole system. Another problem is that existing methods for formation control have difficulty translating complex desired formations into simple individual robot actions. Prior work comparing centralized and distributed swarm coordination reports that centralized control can produce higher-quality decisions, but it also faces significant scalability limitations as swarm size grows because latency, reliability, and coordination overhead become more difficult to manage [2]. This paper presents a decentralized, scalable framework that allows drones to respond to situations based on their immediate sensor data to achieve a common goal – occupying a volume.

II. DESIGN & IMPLEMENTATION

The project was developed in three progressively more realistic stages. We first defined a high-level decentralized controller architecture, then implemented that architecture inside a pure Python sandbox with Matplotlib-based visualization and a simplified quadrotor dynamics model, and finally transferred the validated design into a ROS2 and Gazebo environment. Organizing the work this way made it possible to separate controller design from simulator integration: the theoretical structure of the controller was established first, the control loops were stress-tested in a lightweight prototyping environment, and only then were the same ideas migrated into a robotics middleware and physics simulator.

A. Theory

At a high level, the controller was designed as a hierarchical pipeline that separated motion intent from low-

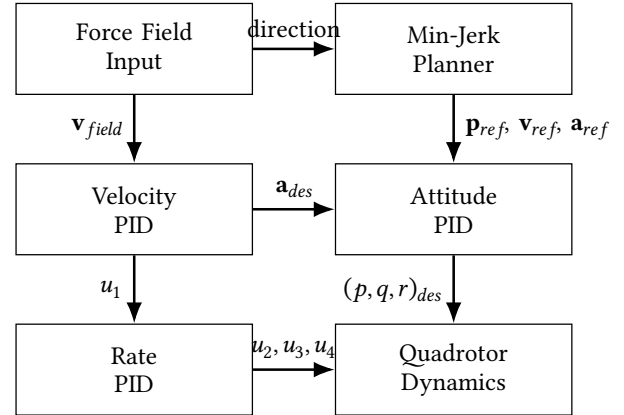


Fig. 1. High-level controller architecture used in the pure Python sandbox and later migrated to the ROS2 and Gazebo implementation. The force field produces motion intent, the minimum-jerk planner smooths that intent into trackable references, and the cascaded PID loops convert those references into thrust and moment commands for the quadrotor dynamics.

level quadrotor actuation. Each robot maintained a 12-state representation [3],

$$[x, y, z, \phi, \theta, \psi, v_x, v_y, v_z, p, q, r],$$

and all robots executed the same decentralized policy. The key idea was that the swarm logic should decide *where the robot ought to move next*, while a cascaded flight controller should decide *how the quadrotor must tilt and generate thrust to realize that motion*. Figure 1 summarizes this control pipeline from the force-field input down to the quadrotor dynamics.

The outermost layer used an artificial force field to produce a desired motion direction. This force field combined three behaviors: neighbor repulsion for collision avoidance, attraction toward a shared goal region, and a ground-avoidance barrier that prevented the vehicle from descending too close to the floor. Rather than directly commanding motors, this layer generated a geometric steering signal that encoded how each robot should react to its local surroundings. Because every robot used the same local rules and no robot was assigned a fixed slot in the formation, the swarm behavior emerged from interaction rather than centralized coordination.

The next layer transformed this geometric direction into a smooth translational command through a minimum-jerk trajectory planner. Conceptually, the force field selected the direction of travel, while the planner attached timing and

speed so that the output became a physically meaningful velocity and acceleration reference instead of an abrupt change in motion.

Below the planner, the flight controller was organized as a cascade of PID loops. The first loop was a velocity controller. It compared the commanded translational velocity to the current vehicle velocity and produced a desired translational acceleration. This loop determined how strongly the quadrotor should accelerate in order to follow the smooth velocity command produced by the planning layer.

The second loop was an attitude controller. Once the desired translational acceleration was known, it was converted into a collective thrust command and desired roll and pitch angles, together with a yaw reference. The attitude controller then drove the vehicle orientation toward those desired angles and generated desired body rates.

The innermost loop was a rate controller. It tracked the desired body rates and generated torque-like roll, pitch, and yaw commands. Functionally, this loop played the role of the low-level actuator interface: it converted rotational tracking errors into the control moments required to reorient the quadrotor.

This hierarchy created a natural path to the later ROS2 and Gazebo implementation: once the controller behaved correctly in a compact mathematical model, the same structure could be transferred to a richer robotics simulation environment with minimal conceptual change.

B. Physics [4]

A quadrotor flies by pushing air down through its rotors. The formula for the thrust generated is:

$$T = k_f \omega^2 \quad (1)$$

Where:

- k_f – thrust coefficient
- ω – angular speed

The rotation of the rotors produces a torque in the opposite direction given by:

$$\tau = k_m \omega^2 \quad (2)$$

This property can be used to produce an interesting application. By making the rotors rotate in pairs—two clockwise and two counterclockwise—and having diagonally opposite rotors rotate in the same direction, we can generate yaw without changing any of the other states.

To describe the motion of the drone, we define a rotation matrix that maps the quadrotor's body frame to the inertial frame. The linear acceleration is determined by the orientation of the thrust vector relative to gravity:

$$m\ddot{\mathbf{r}} = R \begin{bmatrix} 0 \\ 0 \\ \sum T_i \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} - \mathbf{F}_D \quad (3)$$

Where:

- m – total mass of the quadrotor
- g – acceleration due to gravity
- $\mathbf{F}_D$ – translational drag forces (air resistance)

The rotational dynamics follow the Euler equations for a rigid body. The angular acceleration $\dot{\omega}$ depends on the applied control torques and the gyroscopic effects of the rotating frame:

$$\tau = I\dot{\omega} + \omega \times (I\omega) \quad (4)$$

Where I is the inertia matrix and $\omega \times (I\omega)$ represents the Gyroscopic effects that arise in a rotating reference frame.

C. Python Sandbox Implementation

The first implementation of this controller architecture was built as a self-contained Python sandbox so the planning and control logic could be debugged quickly. This environment used Matplotlib for lightweight visualization, making it easy to inspect trajectories, relative spacing, and transient responses without the overhead of a full robotics simulator. The pure Python setting was particularly useful for tuning controller gains, verifying the behavior of the force-field interactions, and checking whether the planner and controller produced qualitatively stable swarm motion.

Within this sandbox, the force-field terms were assembled into the velocity-field command `ext_forces`. The code combined repulsion from nearby neighbors, attraction toward the shared goal volume, and a ground barrier, then added damping to suppress oscillatory motion. This velocity-field command represented the immediate directional intent produced by the swarm logic. The controller then passed that intent into `CascadedController.minimum_jerk_trajectory`, which used a fifth-order polynomial to generate smooth position, velocity, and acceleration references over a segment.

The sandbox then executed the cascaded PID controller numerically. The velocity PID formed a commanded velocity,

$$\mathbf{v}_{cmd} = \mathbf{v}_{ref} + K_p(\mathbf{p}_{ref} - \mathbf{p}) + \mathbf{v}_{field},$$

and converted the resulting tracking error into a desired acceleration. That desired acceleration was mapped into collective thrust and desired roll and pitch commands, which were processed by the attitude PID to generate desired body rates. Finally, the rate PID converted rate errors into the control inputs u_2 , u_3 , and u_4 . Together with collective thrust u_1 , these inputs closed the low-level flight control loop.

The vehicle response was simulated with a simplified quadrotor rigid-body model [3]. This model assumed a standard 12-state quadrotor with Euler-angle attitude representation, thrust applied along the body vertical axis, and rotational dynamics driven by principal-axis inertias. Translational acceleration came from the rotated thrust vector and gravity, while rotational acceleration came from the applied roll, pitch, and yaw moments together with inertial coupling terms. The state was advanced with forward Euler integration at each step. Although this model ignored aerodynamic effects such as rotor flapping, motor lag, and detailed ground

interaction, it captured the dominant relationships between tilt, thrust, velocity, and angular rates. As a result, it was sufficiently rich to validate the controller structure while still remaining simple enough for rapid iteration.

This pure Python implementation therefore served as a bridge between controller theory and full robotics simulation. It provided a controlled environment for validating the force-field planner, the minimum-jerk trajectory shaping, and the cascaded PID loops before any ROS topics, Gazebo plugins, or robot description files were introduced. Once this version demonstrated that the swarm could generate sensible motion and remain stable under the assumed dynamics, the project naturally progressed to Gazebo and ROS2, where the same controller ideas could be tested in a more realistic multi-robot workflow.

D. Gazebo and ROS2 Implementation

The project was implemented as a ROS2 and Gazebo simulation with three aerial robots. The main package, `multi_aerial_gazebo`, includes the robot URDF model, Gazebo world file, launch file, and Python control nodes.

The Gazebo world contains a bounded arena, several static obstacles, and a red spherical target. Each aerial robot is described by a simplified URDF model with a base body, fixed arms, and four propeller visuals. This simplified model allows the project to focus on multi-robot control and obstacle avoidance rather than detailed aerodynamic modeling.

The launch file `three_aerial_gazebo.launch.py` starts Gazebo, spawns three robots, and launches the ROS2 nodes. The robots are spawned at randomized collision-free ground positions while maintaining safe distance from obstacles and from each other. A start signal on the topic `/start_flight` is used to trigger takeoff, so the robots remain still until the user begins the demo.

A custom state interface node reads robot poses and velocities from Gazebo and publishes a 12-state vector for each robot:

$$[x, y, z, roll, pitch, yaw, v_x, v_y, v_z, p, q, r]$$

These states are published on topics such as `/aerial_robot_1/state12`. The node also receives body-frame velocity commands from topics such as `/aerial_robot_1/cmd_vel_body` and applies them back to the Gazebo models.

The controller node uses PID position control to move each robot toward the shared target. To avoid collisions, a force field is added to push robots away from static obstacles and from each other. The final command is computed as

$$\mathbf{v}_{cmd} = \mathbf{v}_{PID} + \mathbf{v}_{APF}$$

and then converted into a body-frame geometry_msgs/Twist command. This structure separates the Gazebo interface from the control algorithm, making the system easier to test, debug, and extend.

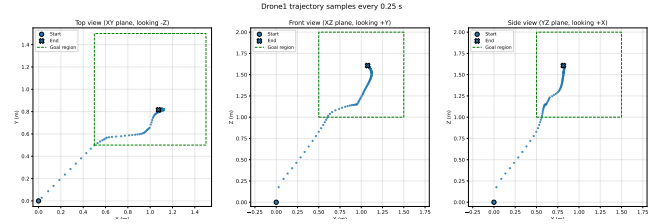


Fig. 2. Sampled trajectory of Drone 1 in the Python sandbox. Widely spaced samples appear along long straight segments, while tightly clustered samples appear near sharp corners, demonstrating the smoothing and speed modulation produced by the minimum-jerk trajectory planner.

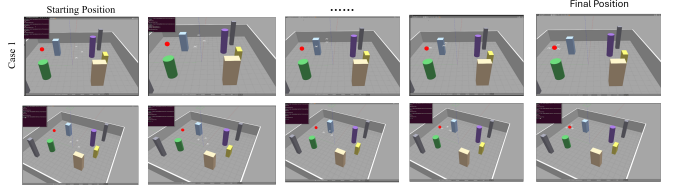


Fig. 3. Gazebo simulation result for the three aerial robots.

III. RESULTS & DISCUSSION

A. Python Sandbox Results

The Python sandbox results show that the controller stack can produce smooth and physically reasonable motion before moving to the more complex ROS2 and Gazebo environment. In particular, the sampled trajectory of one drone demonstrates that the minimum-jerk planner is behaving as intended. Figure 2 shows that successive sample points are spaced farther apart along long, nearly straight portions of the path, indicating that the vehicle is moving efficiently through low-curvature segments. In contrast, the sample points become more tightly clustered around sharper turns, which indicates that the planner slows the vehicle as the path curvature increases.

More broadly, this plot provides evidence that the Python sandbox successfully integrates the force-field guidance, the minimum-jerk planner, and the cascaded PID controller into a stable closed-loop system. The drone does not simply jump between waypoints; instead, it follows a continuous path with speed naturally modulated by the geometry of the motion. This result was important because it confirmed that the planning and control architecture was functioning correctly in a lightweight simulation before being transferred to the higher-fidelity Gazebo and ROS2 framework.

B. Gazebo Results

We randomized the starting positions of the three aerial robots within the bounded arena. The resulting Gazebo scene is shown in Figure 3. The path-planning algorithm successfully guides the robots to the red target while maintaining obstacle avoidance throughout the maneuver.

C. Future Steps

In the future, simulations can be improved by introducing more realistic noise and uncertainty into the Gazebo envi-

ronment. Currently, the robot’s state and control commands are relatively idealized, but real-world robots are typically subject to sensor noise, actuator delays, imperfect velocity tracking, and small disturbances from the environment. By introducing noise into position, velocity, or sensor readings, controllers can be tested under conditions that more closely resemble real physical systems.

Another potential improvement is modeling external disturbances, such as wind or small random forces. This would increase the difficulty of controlling aerial robots and help evaluate whether the PID controller is sufficiently robust when the robot fails to move exactly as commanded.

Obstacle avoidance methods can also be improved. Although the force field method currently used is simple and effective, it may get stuck in local minima or produce unstable motion near obstacles. Future research could compare this approach with more advanced planning methods, such as A^* search, RRT, or model predictive control.

Finally, simulation results can be transferred to real robot platforms. After testing in Gazebo, the same ROS2 topic structure and controller logic can be adapted to real hardware. This will enable the project to evaluate the actual performance of the simulated controller in a real-world environment.

IV. APPENDIX

Our implementation of HIVE is available online at https://github.com/Peteroos/Robotics2_final_project

A. Team Contribution

Eric Cardozo: Created a 2D model in MATLAB for the quadrotor, worked the physics of flight, attempted to relate the voltage produced by the hardware to the resultant torque in the rotors, contributed to the writing of the final report.

Peter Chen: Primarily led the ROS2 and Gazebo simulation effort, including building the simulation world, integrating PID control for the aerial robots, implementing the planning algorithm in Gazebo, and recording the demonstration video. He also contributed to writing the final report.

Matthew McCall: Contributed to the initial controller implementation in Python and Matplotlib. This included tuning PID constants over the course of multiple simulations, and generating the plots that demonstrate minimum-trajectory planning and obstacle avoidance. Matthew also contributed to the writing and formatting of the final report.

REFERENCES

- [1] M. Abdelkader, S. Güler, H. Jaleel, and J. S. Shamma, “Aerial swarms: Recent applications and challenges,” *Current Robotics Reports*, vol. 2, no. 3, pp. 309–320, 2021.
- [2] J. Hu, A. Lanzon, M. Schilling, and F. Iida, “The dilemma of centralized and decentralized control in multi-robot systems: A review of the trade-offs,” *arXiv preprint arXiv:1805.01786*, 2018.
- [3] E. Yel, “Quadrotor state dynamics hover control solution,” 2026, course material distributed via Rensselaer Polytechnic Institute LMS.
- [4] —, “Lecture 15, how does a quadrotor fly?” 2026, course material distributed via Rensselaer Polytechnic Institute LMS.